

System Design Basics

Lesson 3: High Level Architecture Patterns

High Level Architecture Patterns - Study Notes

Key Concepts

1. **Monolithic Architecture** – A single, tightly coupled codebase that contains all application functionality.
2. **Microservices Architecture** – A collection of small, independently deployable services that communicate over well defined APIs.
3. **Serverless Architecture** – Event driven, function as a service (FaaS) model where the cloud provider manages the underlying infrastructure.
4. **Service Boundaries & Contracts** – Defining clear interfaces (REST, gRPC, message queues) that separate responsibilities.
5. **Deployment & Scaling Models** – How each pattern handles scaling, deployment frequency, and fault isolation.

Summary

High level architecture patterns shape how we structure, deploy, and evolve software systems. A **monolith** bundles all features into one executable, simplifying development but making scaling and maintenance harder as the codebase grows. **Microservices** break the application into focused services that can be developed, deployed, and scaled independently, improving agility and resilience at the cost of added operational complexity. **Serverless** abstracts the server layer entirely, letting developers focus on code while the cloud provider automatically scales functions in response to events, but it introduces constraints such as cold start latency and statelessness.

Choosing the right pattern depends on factors like team size, product velocity, operational expertise, and the need for fault isolation. Often, real world systems blend patterns—for example, a core monolith with a microservice based authentication layer, or a microservice architecture that leverages serverless functions for specific event driven tasks.

Important Points

- **Monolith**
 - Single deployment unit; easy CI/CD but hard to scale horizontally.
 - Tight coupling can lead to “big ball of mud” if not modularized.
 - Suitable for small teams or early stage MVPs.
- **Microservices**
 - Each service owns its own database (Database per Service pattern) to avoid tight coupling.
 - Requires robust inter service communication (synchronous REST or asynchronous message queues).
 - Observability (logging, tracing, metrics) is critical due to distributed nature.
 - Deployment pipelines must handle many services; container orchestration (K8s) often used.

- **Serverless**

- Functions are stateless; external state stored in databases or object stores.
- Pay per execution pricing; ideal for sporadic workloads.
- Cold starts can add latency; mitigate with provisioned concurrency or keep warm strategies.
- Suitable for event driven tasks (image processing, webhook handlers, scheduled jobs).
 - **Common Trade offs**
- **Complexity vs. Flexibility**: Microservices add operational overhead but provide fine grained scaling.
- **Latency vs. Cost**: Serverless can be cheaper but may incur higher latency for frequent calls.
- **Team Autonomy vs. Governance**: Microservices empower teams but require governance to avoid “service sprawl.”

Examples

1. Microservice for User Authentication

- **Service**: `AuthService`
- **Responsibilities**: User registration, login, JWT issuance, password reset.
- **API**: REST endpoints (`/register`, `/login`, `/refresh`, `/reset-password`).
- **Data Store**: Dedicated PostgreSQL database with user table; no direct access from other services.
- **Communication**: Other services validate JWTs locally; optionally subscribe to a message bus for user created events.
- **Deployment**: Docker container, deployed to Kubernetes with horizontal pod autoscaling.

Why it's a good example: It encapsulates a single business capability, can evolve independently, and demonstrates clear API contracts and data ownership.

2. Serverless Image Resizer

- **Trigger**: Object storage event (e.g., S3 `ObjectCreated`).
- **Function**: `resizeImage(event)`
- Reads the original image from S3.
- Generates multiple sizes (thumbnail, medium, large).
- Stores resized images back to S3.
 - **Runtime**: AWS Lambda (Node.js) or Azure Functions (Python).
 - **Scaling**: Automatic; the cloud provider spins up instances per event.
 - **Cost**: Pay only for the compute time and storage I/O.

Why it's a good example: Highlights the event driven, stateless nature of serverless, and shows how to offload compute-intensive tasks without managing servers.

Quick Review

1. What are the main advantages of a monolithic architecture compared to microservices?
2. Explain the “Database per Service” pattern and why it matters in microservices.
3. What causes cold start latency in serverless functions, and how can it be mitigated?
4. List two key observability practices required for a microservices deployment.
5. When might a team choose a hybrid approach (e.g., monolith + microservices)?

Further Reading

- **Microservices Patterns** – Chris Richardson
- **Building Serverless Applications** – Amazon Web Services (AWS) whitepapers
- **Monolith to Microservices Migration** – Martin Fowler's blog series
- **Observability in Distributed Systems** – Google Cloud Architecture Center
- **Event Driven Architecture** – Microsoft Docs (Azure Event Grid, Service Bus)

Feel free to dive into these resources to deepen your understanding of each pattern and how they can be applied in real projects!

